ComPDF Generation

Java SDK

Developer Guide

Version 1.3.0

2026-10-14

Java integration, email conversion and print configuration

Clickable contents and chapter bookmarks

Contents

1 Overview . . . 4
1.1 ComPDF Generation SDK . . . 4
1.2 Key Features . . . 4
1.2.1 Generate PDF from HTML . . . 4
1.2.2 Generate PDF Using Layout API . . . 4
1.2.3 Convert Email to HTML and PDF . . . 4

2 Get Started . . . 5
2.1 Requirements . . . 5
2.1.1 Obtaining a License Key . . . 5
2.1.2 System Requirements . . . 5
2.2 PDF Generation SDK Package Structure . . . 5
2.3 Apply the license Key . . . 6
2.3.1 Copying the License Key . . . 6
2.3.2 Apply the License Key . . . 6
2.4 How to Run a Demo . . . 6
2.5 How to Use PDF Generation SDK in Java Applications . . . 7
2.6 Samples . . . 8
2.6.1 HTML to PDF Examples . . . 8
2.6.2 Layout API Examples . . . 8
2.6.3 Email and Print Configuration Examples . . . 8

3 Guides . . . 9
3.1 Overview . . . 9
3.2 Generate PDF from HTML . . . 10
3.2.1 Overview . . . 10
3.2.2 Generate PDF from HTML template . . . 10
3.2.3 Convert HTML to PDF . . . 16
3.2.4 Page Margins . . . 17
3.2.5 Headers and Footers . . . 18
3.3 Generate PDF Using Layout API . . . 19
3.3.1 Introduction . . . 19
3.3.2 Core Concepts . . . 19
3.3.3 Quick Start . . . 21
3.3.4 Text and Paragraphs . . . 21
3.3.5 Tables . . . 22
3.3.6 Lists . . . 24

3.3.7 Images . . . 25
3.3.8 Style Management . . . 26
3.3.9 Box Model . . . 26
3.3.10 Positioning . . . 27
3.3.11 Background . . . 28
3.3.12 Separators . . . 28
3.3.13 Font Management . . . 29
3.3.14 Complete Examples . . . 30
3.3.15 Best Practices . . . 33
3.3.16 Developer FAQs . . . 34
3.4 Convert Email to HTML and PDF . . . 34
3.4.1 Formats and Initialization . . . 34
3.4.2 Convert an Email File to PDF . . . 35
3.4.3 Export HTML and Write PDF in Separate Steps . . . 35
3.4.4 Options and Cancellation . . . 36
3.4.5 Stream and Memory Input . . . 37
3.4.6 Result Data and Status Codes . . . 39
3.4.7 Resource Lifetime and Concurrency . . . 40
3.4.8 Email Sample . . . 40

4 Technical support . . . 42
4.1 Report issues . . . 42
4.2 Contact information . . . 42

1 Overview

ComPDF Generation for Java is a powerful PDF generation library that provides multiple ways to generate high-quality PDF documents. Whether through HTML conversion or direct code construction, it can meet your various PDF generation needs.

1.1 ComPDF Generation SDK

ComPDF Generation for Java supports Linux.

1.2 Key Features

PDF Generation component offers the following features:

1.2.1 Generate PDF from HTML

- **HTML to PDF**：Supports converting HTML strings, files, or streams into PDF documents.
- **Template Rendering**：Supports dynamically filling HTML templates using JSON data and rendering them into PDF documents.
- **Custom Configuration**：Configure page size, page margins, base URI, fonts, and header/footer templates.
- **Font Management**：Supports custom fonts and can integrate system fonts.

1.2.2 Generate PDF Using Layout API

- **Programmatic Document Construction**: Directly create and control every element of PDF documents through code.
- **Rich Element Types**: Supports text, paragraphs, tables, lists, images, separators, and various other elements.
- **Flexible Layout Control**: Supports block-level elements, inline elements, and absolute positioning, providing precise layout control.
- **Complete Style System**: Supports reusable styles, box model (margin, padding, border), background, fonts, etc.
- **Flowing Layout**: Automatically handles pagination and content reflow, supports table headers and footers repeating across multiple pages.
- **Tree-Structured Document**: Adopts an intuitive hierarchical structure to organize document content, easy to understand and maintain.

The Layout API is particularly suitable for scenarios that require precise control of document layout, dynamic generation of complex structured documents, or deep integration with existing code logic.

1.2.3 Convert Email to HTML and PDF

- **EML and MSG Input**: Convert email files, byte arrays or streams to HTML or PDF.
- **HTML Package and Attachments**: Export the selected email body with its resources and available attachments.
- **Conversion Control**: Inspect status, diagnostics and body provenance; configure byte budgets and cooperative cancellation.

2 Get Started

2.1 Requirements

Before starting, please make sure that you have already met the following prerequisites.

2.1.1 Obtaining a License Key

ComPDF provides two types of license key: 30-day free trial license and commercial license.

How to Get Free Trial License

Contact our sales team and we'll send you a 30-day free trial license for PDF Generation SDK.

How to Get Commercial License

PDF Generation SDK is a commercial SDK that requires a license for application release. Any documents, sample code, or source code distribution from the released package of ComPDFKit PDF SDK to any third party is prohibited.

To get commercial licenses for advanced features, custom requirements, or quote inquiries, feel free to contact our sales team.

2.1.2 System Requirements

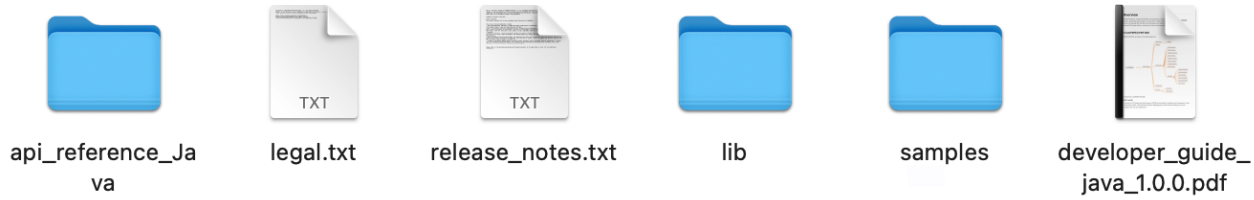
Programming Environment	Java JDK 1.8 and higher
**Supported Platforms **	**Linux (x86)**

2.2 PDF Generation SDK Package Structure

The package of ComPDF Generation SDK for Java includes the following files:

- **"samples"** - Java sample projects.
  - **"ConvertToPdf"** - Convert HTML to PDF.
  - **"RenderTemplate"** - Populate an HTML template with JSON.
  - **"RenderAndConvert"** - Render a template and convert it to PDF.
  - **"PrintOptionsTest"** - Configure page margins and headers/footers.
  - **"FamilyHeaderFooterTest"** - Convert the family HTML example with cover-page and numbering settings.
  - **"EmailConvertTest"** - Parse EML/MSG and demonstrate HTML packages and PDF output from files and streams.
  - **"TestFiles"** - Sample inputs, including `email` and `family_html`.
  - **"lib"** - The sample dependency `generation-1.3.0.jar`.
- **"generation-1.3.0.jar"** - The SDK JAR, including the matching Linux native library.
- **"API Reference"** - Java API reference; open `index.html`.
- **"develop_guide_java_en.pdf"** - English developer guide.
- **"release_notes.txt"** - Release information.

- **"legal.txt"** - Legal and copyright information.



2.3 Apply the license Key

If you haven't get a license key, please check out how to obtain a license key.

PDF Generation SDK currently supports two authentication methods to verify license keys: online authentication and offline authentication.

Learn about: What is the authentication mechanism of ComPDFKit's license? What are the differences between Online Authentication and Offline Authentication?

2.3.1 Copying the License Key

Accurately obtaining the license key is crucial for the application of the license.

1. In the email you received, locate the `XML` file containing the license key.
2. Open the `XML` file.

Offline License:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<license version="1">
<platform>linux</platform>
<starttime>xxxxxxxx</starttime>
<endtime>xxxxxxxx</endtime>
<key>LICENSE_KEY</key>
</license>
```

3. Copy the **LICENSE_KEY** from the `<key>LICENSE_KEY</key>` field.

2.3.2 Apply the License Key

You can authenticate in the following ways:

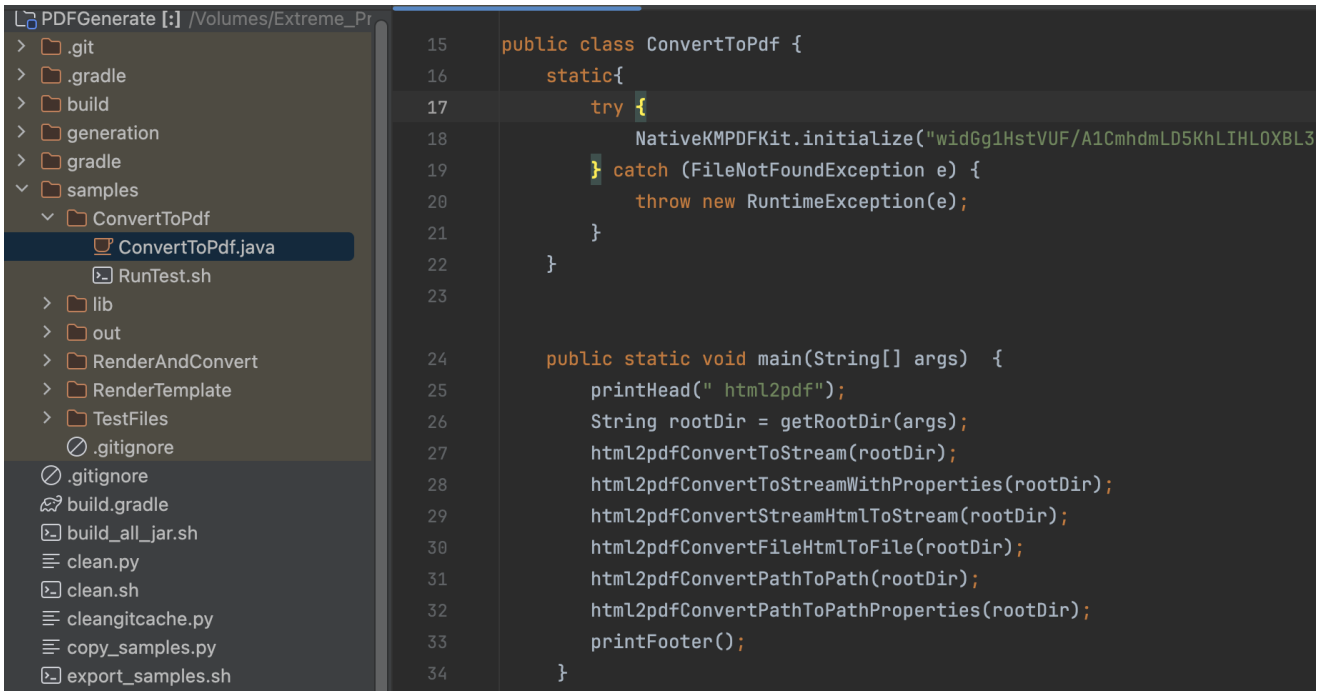
```
NativeComPDF.initialize("LICENSE_KEY", "your deviceId", "", Constants.LIB_DIR_NAME);
```

2.4 How to Run a Demo

PDF Generation SDK for Java provides multiple demos for developers to learn how to call the SDK on Java applications. You can find them in the ***"samples"*** folder. In this guide, we take ***"ConvertToPdf"*** as an example to show how to run it on Java applications.

1. Import the ***"samples"*** project on IDE.
2. Navigate to `samples/ConvertToPdf` and run its `RunTest.sh` script:

```
./RunTest.sh
```

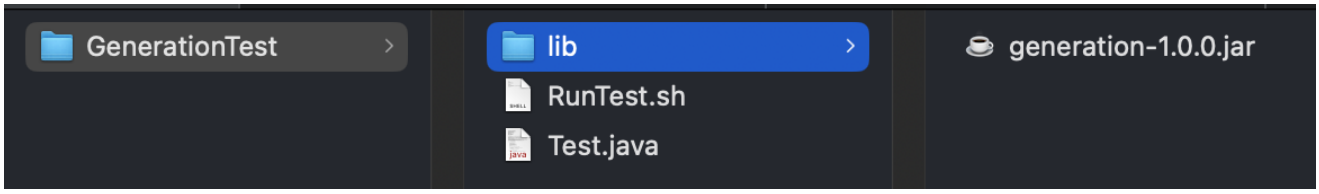


Note: *This is a demo project, presenting completed PDF Generation SDK functions. The functions might be different based on the license you have purchased. Please check that the functions you choose work fine in this demo project.* The output files will be in `/samples/out`.

2.5 How to Use PDF Generation SDK in Java Applications

This section will help you to quickly get started with PDF Generation SDK to make a Java application with step-by-step instructions.

Java project struct:



1. Copy generation-1.3.0.jar from the PDF Genenation SDK to your project directory.
2. Create RunTest.sh file in Java project directory:

```
#Test is the name of the class to be executed, which is the class that contains the main method in t
    he Java project.
TEST_NAME=Test
#The name of the jar file provided in the SDK.
JAR_NAME=generation-1.3.0.jar
#Java project root dictory
ROOTDIR=$(pwd)
#Jar package path of the generation lib.
LIB_DIR=$(pwd)/lib
#Pass jar package path，and compile java file.
javac -cp $LIB_DIR/$JAR_NAME *.java
java -Djava.library.path=LIB_DIR -cp .:$LIB_DIR/$JAR_NAME $TEST_NAME $ROOTDIR
#Delete the unnecessary intermediate generated files.
rm -rf pdfgeneration
rm -rf *.class
```

3. In Terminal，turn to Java project directory, and execute `./RunTest.sh`.

2.6 Samples

The Samples use preset parameters to call the API of PDF Generation SDK for Java for each function without UI interaction or parameter settings. This is achieved through modular code examples. Not only does it demonstrate HTML to PDF functionality, but it also provides complete examples of creating PDF documents directly using the Layout API.

These projects not only demonstrate the best practices for each function but also provide detailed introductions. The impact of each function on PDF documents can be observed in the output directory. With the help of the Samples, you can quickly learn how to use the functions you need and apply them to your projects.

2.6.1 HTML to PDF Examples

Name	Description
ConvertToPdf	Convert HTML to PDF.
RenderTemplate	Combine HTML template with JSON and generate complete HTML.
RenderAndConvert	Combine HTML template with JSON data to generate complete HTML, and convert the complete HTML to PDF.

2.6.2 Layout API Examples

Name	Description
CreateParagraph	Create paragraphs with various text formatting options.
CreateTable	Create tables with headers, footers, and styled cells.
CreateList	Create ordered and unordered lists with multiple styles.

2.6.3 Email and Print Configuration Examples

Name	Description
EmailConvertTest	Parse EML/MSG, export an offline HTML package, and generate PDF from files or streams.
PrintOptionsTest	Set/read page margins and header/footer options.
FamilyHeaderFooterTest	Apply cover-page skipping and page-number templates to the family HTML example.

3 Guides

If you're interested in the features mentioned in Overview section, please go through our guides to quickly add PDF generation to your application. The following sections list some examples to show you how to add PDF generation functionalities to Java apps using the APIs of PDF generation SDK for Java.

3.1 Overview

Using the PDF Generation Java library, you can achieve fast and bulk PDF generation in your application. We provide different methods for PDF generation, and you can choose the one that best fits your business scenario.

1. **Generating PDF from HTML templates**: Simply create an HTML template once, and then use JSON data to populate the template for bulk generation of different PDFs while retaining the original page layout. This method supports inserting text, images, and dynamic tables, making it ideal for generating complex and elegant PDFs, such as reports, invoices, letters, and contracts.
2. **Converting HTML to PDF**: Standard HTML files or HTML strings can be converted into PDF files, suitable for quickly converting existing HTML content to PDF documents.
3. **Generating PDF through Layout API**: Directly create and control every element of PDF documents through code, providing the finest level of control. This method allows you to completely build PDF documents from scratch programmatically, making it ideal for scenarios that require precise control of document layout, dynamic generation of complex structured documents, or deep integration with existing code logic. The Layout API supports a rich set of element types (text, paragraphs, tables, lists, images, etc.) and provides a complete style system and flexible layout control.
4. **Converting email to HTML or PDF**: Parse EML/MSG, inspect the selected body and attachments, export an offline HTML package, or render the email to PDF. See Section 3.4.

Main Capabilities of HTML PDF Generation:

1. Fast speed, capable of meeting the demand for bulk PDF generation.
2. Perfectly restores template layout.
3. Rich support for HTML tags and CSS styles.
4. Supports text reflow, maintaining a perfect page layout.
5. Supports dynamic tables.

Main Capabilities of Layout API PDF Generation:

1. **Complete Programmatic Control**: Precisely control the position, size, and style of each element in the document.
2. **Rich Element Types**: Supports text, paragraphs, tables, lists, images, separators, and various other elements.
3. **Flexible Layout System**: Supports block-level elements, inline elements, and absolute positioning.
4. **Complete Style System**: Supports reusable styles, box model, background, fonts, etc.
5. **Flowing Layout**: Automatically handles pagination and content reflow, supports table headers and footers repeating across multiple pages.
6. **Tree-Structured Document**: Adopts an intuitive hierarchical structure to organize document content.

Start your experience now!

3.2 Generate PDF from HTML

3.2.1 Overview

Supports dynamic PDF file generation through HTML templates, or generating PDF files from standard HTML files or HTML strings.

To use the PDF generation feature through HTML templates, you need to prepare the following:

- **Template:** Design your template in HTML format.
- **Replacement data:** Provide the filling data in JSON format.

ComPDF will merge the data you provide with the template to efficiently generate PDF files. During the filling process, you can also use conditional statements, loops, and more, allowing you to implement more flexible filling rules. The remaining tasks are taken care of by ComPDF.

3.2.2 Generate PDF from HTML template

Overview of the overall process

The Generation from HTML feature requires an HTML template and structured data for filling the template, which is JSON data, to generate a standard HTML. Then, by using the corresponding interface of Generation from HTML, you can convert the standard HTML into a PDF document.

1. Build HTML template

The template is an HTML file, and a matching JSON data file is also required. The JSON data is a standard JSON file. The HTML template file needs to be combined with the JSON data to generate a standard HTML. The process of generating HTML by combining JSON data and the HTML template supports variables, if statements, for loops, arithmetic operations, and common functions.

HTML template example:

```
<!doctype html>
<html>
<body style="margin: 0; padding: 0;">
<table cellpadding="0" cellspacing="0" width="100%" style="border-collapse: collapse; margin: 0 auto
    ">
  <tr>
    <td style="padding-top: 40px;">
      <img
          src="{% if compdfkit %}images/logo.png{% else if compdfkit2 %}images/logo2.png{% else %}im
              ages/logo3.png{% endif %}"
          alt="PDF Technologies"
          width="203"
          height="50">
    </td>
    <td style="padding-top: 40px; text-align: right;">
      <img src="images/invoice.png" alt="invoice" width="132">
      <td style="border: 1px solid #8A8EA8; padding: 2px 10px;">{{billNo}}</td>
    </td>
  </tr>
</table>
<table cellpadding="0" cellspacing="0" width="100%" style="border-collapse: collapse;">
  <!-- {% set totalprice = 0 %} -->
  <!-- {% for item in items %} -->
  <tr style="height:64px;color:#333;">
    <td style="width:70px;height:70px;border-bottom: 2px solid #F3F3F3; padding-left: 10px; text-ali
        gn: left;">
      {{ item.index }}
    </td>
    <td style="width: 160px;border-bottom: 2px solid #F3F3F3; font-size: 13px; line-height: 15px; co
```

```
        lor: #121E3F;text-align:left;">
      <div>
        <p style="font-size: 13px; line-height: 15px; font-weight: bold; color: #121E3F; margin: 0;"
           >
          {{ item.product }}
        </p>
        <p style="font-size: 11px; line-height: 12px; padding-top: 4px; color: #5F6881; margin: 0;">
          {{ item.startDateAndEndDate }}
        </p>
      </div>
    </td>
    <td style="width: 130px;border-bottom: 2px solid #F3F3F3; font-size: 13px; line-height: 15px; co
        lor: #121E3F;text-align:left;">
      {{ item.paymentCycle }}
    </td>
    <td style="width: 100px;border-bottom: 2px solid #F3F3F3; font-size: 13px; line-height: 15px; co
        lor: #121E3F;text-align:left;">
      {{ item.gearLevel }}
    </td>
    <td style="width: 100px;text-align:left;border-bottom: 2px solid #F3F3F3; font-size: 13px; line-
        height: 15px; color: #121E3F;">
      $ {{ item.price }}
    </td>
  </tr>
  <!-- {% set totalprice = totalprice + item.price %} -->
  <!-- {% endfor %} -->
  <tr>
    <td style="width:70px;"></td>
    <td style="width:160px;"></td>
    <td colspan="2" style="width: 100px; text-align:right; font-weight: bold; font-size: 16px; line-
        height: 18px; color: #121E3F; padding-right: 72px;">Total:</td>
    <td style="width: 100px; text-align:left; font-weight: bold; font-size: 16px; line-height: 18px;
         color: #121E3F;">$ {{round(totalprice * (1 + tax * 0.01), 2)}}</td>
  </tr>
</table>
</body>
</html>
```

As shown in the example above, the HTML template contains some non-HTML standard content, such as `{% if compdfkit %}`, `{{billNo}}`, `<!-- {% set totalprice = totalprice + item.price %} -->`, and `<!-- {% for item in items %} -->`. This non-standard HTML content includes variables, if statements, for loops, arithmetic operations, and common functions, among these types.

Variables

```
<td style="border: 1px solid #8A8EA8; padding: 2px 10px;">{{billNo}}</td>
```

The above `{{billNo}}` is a variable, which needs to be enclosed in `{{ }}` and corresponds to an entry named `billNo` in the JSON file.

If the corresponding name does not exist in the JSON file, it needs to be defined first. Below is an example of defining a variable.

```
<!-- {% set totalprice = 0 %} -->
```

In the example template, the line of code above defines a variable named `totalprice`. Similar to the previous `billNo` variable, it is used as follows after being enclosed in `{{ }}`: `{{totalprice}}`.

Arithmetic operations

```
<!-- {% set totalprice = totalprice + item.price %} -->
```

The line of code above performs calculations on `totalprice`, supporting addition, subtraction, multiplication, and division.

if statements

```
<img
     src="{% if compdfkit %}images/logo.png{% else if compdfkit2 %}images/logo2.png{% else %}images/
         logo3.png{% endif %}"
     alt="PDF Technologies"
     width="203"
     height="50">
```

The `{% if compdfkit %} … {% else if compdfkit2 %}...{% else %}...{% endif %}` is a complete `if statement` structure, where `else if` and `else` are optional. The variables `compdfkit` and `compdfkit2` correspond to entries with the same names in the JSON file.

for loops

```
<!-- {% set totalprice = 0 %} -->
<!-- {% for item in items %} -->
                .........
      <!-- {% set totalprice = totalprice + item.price %} -->
<!-- {% endfor %} -->
```

`{% for item in items %}...{% endfor %}` is a structure for a for loop. Here, `items` is an array type of data in the JSON file.

Common functions

```
<td style="width: 100px; text-align:left; font-weight: bold; font-size: 16px; line-height: 18px; col
    or: #121E3F;">$ {{round(totalprice * (1 + tax * 0.01), 2)}}</td>
```

In the code, `round` is a function that can take variables as parameters. `totalprice` and `tax` are variables. The entire function is enclosed in `{{ }}`.

2. Prepare JSON data

Prepare JSON data in standard format to combine with the HTML template and fill the data into the template to generate standard HTML. Example of JSON data:

```
{
    "address": "456 Oak Avenue",
    "billNo": "2345678901",
    "company": "TechCorp Ltd.",
    "country": "Canada",
    "email": "alice.smith@techcorp.com",
    "firstName": "Alice",
    "gearLevel": "Pro",
    "compdfkit": false,
    "compdfkit2": true,
    "invoiceDate": "2025-01-15",
    "lastName": "Smith",
    "paymentCycle": "Yearly",
    "price": "1200",
    "province": "Ontario",
    "startDateAndEndDate": "2025-01-15 to 2025-02-15",
    "tax": 8,
    "zip": "K1A 0A6",
    "items": [
        {
            "index": 1,
            "product": "ComPDFKit Pro",
            "startDateAndEndDate": "2025-01-15 to 2025-02-15",
            "paymentCycle": "Yearly",
            "gearLevel": "Pro",
            "price": 800
        },
        {
            "index": 2,
```

```
            "product": "ComPDFKit API",
            "startDateAndEndDate": "2025-01-15 to 2025-02-15",
            "paymentCycle": "Monthly",
            "gearLevel": "Basic",
            "price": 400
        }
    ]
}
```

3. Fill the template to generate the PDF

Convert JSON file combined with HTML template to PDF:

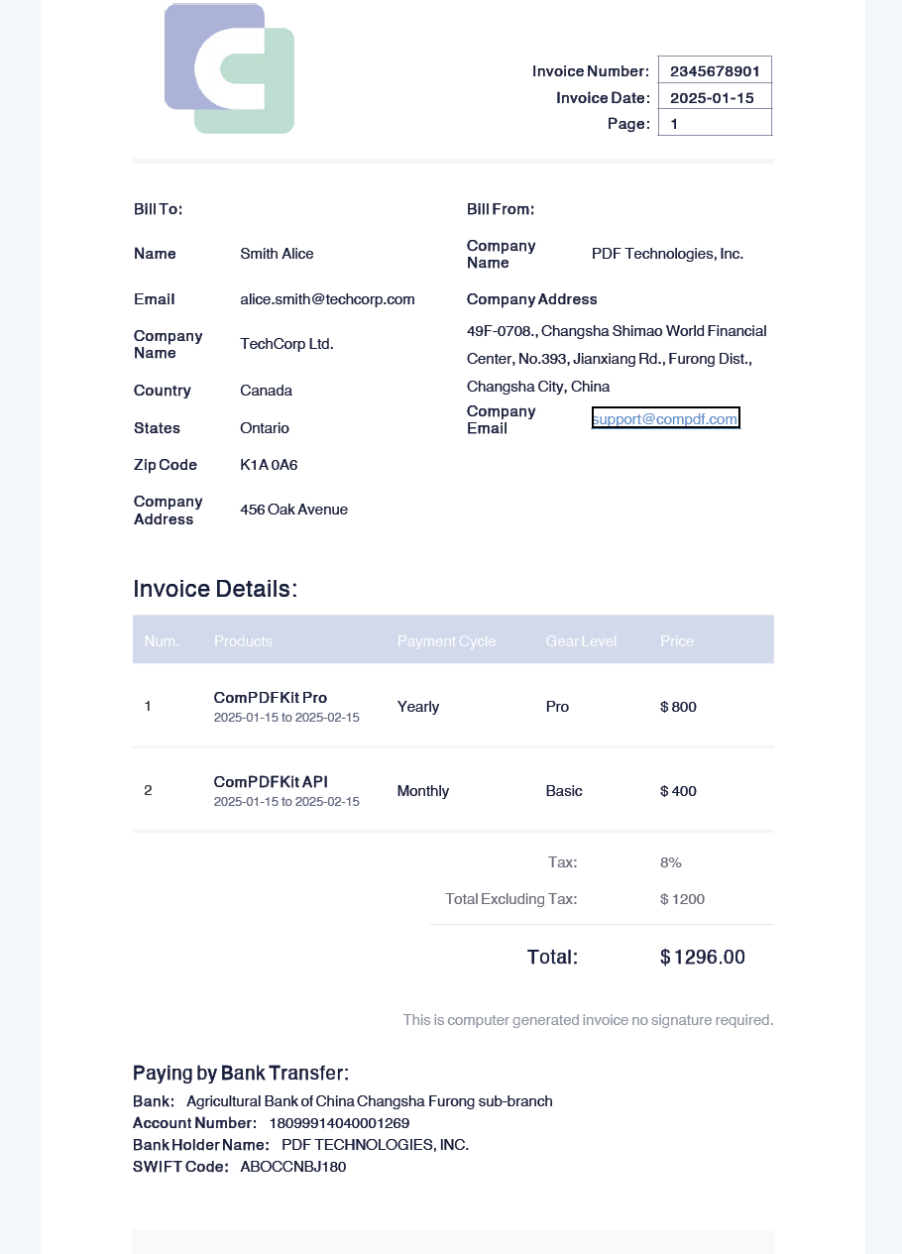
```
//HTML template file path
String templatePath = "/template_1.html";
//json file path
String jsonPath = "/data1.json";
//Output HTML path
String outHtmlPath = "/outHtml.html";
//Output PDF path
String outPdfPath = "/outPdf.pdf";

//Combine the JSON file with the HTML template to generate HTML content
CHtmlConverter.CErrorCode id = CHtmlConverter.HtmlRenderTemplate(templatePath, jsonPath, outHtmlPath
    );
//Convert HTML to PDF
id = CHtmlConverter.ConvertToPdf(outHtmlPath, outPdfPath);
```

In the example，`HtmlRenderTemplate` and `ConvertToPdf` these two steps can be replaced by the interface`RenderAndConvertToPdf`.

```
//HTML template file path
String templatePath = "/template_1.html";
//json file path
String jsonPath = "/data1.json";
//Output PDF path
String outPdfPath = "/outPdf.pdf";
CHtmlConverter.CErrorCode id = CHtmlConverter.RenderAndConvertToPdf(templatePath, jsonPath, outPdfPa
    th);
```

4. Generated PDF example



Invoice Number: 2345678901
Invoice Date: 2025-01-15
Page: 1

Bill To:

Name	Smith Alice
Email	alice.smith@techcorp.com
Company Name	TechCorp Ltd.
Country	Canada
States	Ontario
Zip Code	K1A 0A6
Company Address	456 Oak Avenue

Bill From:

Company Name	PDF Technologies, Inc.
Company Address	49F-0708., Changsha Shimao World Financial Center, No.393, Jianxiang Rd., Furong Dist., Changsha City, China
Company Email	support@compdf.com

Invoice Details:

Num.	Products	Payment Cycle	Gear Level	Price
1	ComPDFKit Pro 2025-01-15 to 2025-02-15	Yearly	Pro	$ 800
2	ComPDFKit API 2025-01-15 to 2025-02-15	Monthly	Basic	$ 400

Tax:	8%
Total Excluding Tax:	$ 1200
Total:	$ 1296.00

This is computer generated invoice no signature required.

Paying by Bank Transfer:

Bank: Agricultural Bank of China Changsha Furong sub-branch
Account Number: 18099914040001269
Bank Holder Name: PDF TECHNOLOGIES, INC.
SWIFT Code: ABOCCNBJ180

Configuration Properties When converting HTML to PDF, you can configure certain properties such as base URI, page size, and fonts.

```
//HTML template file path
String templatePath = "/test/template_1.html";
//json file path
String jsonPath = "/test/data1.json";
//Output HTML path
String outHtmlPath = "/test/outHtml.html";
//Output PDF path
String outPdfPath = "/test/outPdf.pdf";

//Combine the JSON file with the HTML template to generate HTML content
CHtmlConverter.CErrorCode id = CHtmlConverter.HtmlRenderTemplate(templatePath, jsonPath, outHtmlPath
    );

//Use properties when conventing
CConverterProperties properties = new CConverterProperties();
//Set page size
CSize size = new CSize(400, 600);
```

```
properties.setPageSize(size);
//Set font directory
CFontProvider fontProvider = new CFontProvider();
fontProvider.addDirectory("/test/fontDir/");
properties.setFontProvider(fontProvider);

//Convert HTML to PDF with Properties
id = CHtmlConverter.ConvertToPdf(outHtmlPath, outPdfPath, properties);
```

The example uses `fontProvider.addDirectory("/test/fontDir/")`. You should provide some font files in the `/fontDir/` directory, and the text in the HTML file can use these provided fonts. Below is part of the HTML example code, where the text inside the span uses the `Sarabun` font. `Sarabun` is a provided font, and the corresponding font files should be placed in the appropriate directory.

```
<style data-dito-stylesheet-name="setFonts">@font-face {
    font-family: "Sarabun";
}
@font-face {
    font-family: "Sarabun";
    font-style: italic;
}
@font-face {
    font-family: "SarabunThin";
}
@font-face {
    font-family: "SarabunsemiBold";
}</style>

<div style="margin-left:0in;margin-top:0pt;margin-bottom:0pt">
  <span style="font-family:Sarabun">
    <span style="color:black">
      <span style="font-size:26px">(PEA GECC Internal Audit)</span>
    </span>
  </span>
</div>
```

During the conversion, the HTML template and JSON can be a content String, InputStream, File object, or file path. The generated PDF can be an OutputStream, File object, or file path. The previous example used file paths, while the following are examples using other types.

Use InputStream and OutputStream:

```
String templatePath = "/test/template_1.html";
String jsonPath = "/test/data1.json";
String outPdfPath = "/test/outPdf.pdf";
try {
    InputStream htmlIn = new FileInputStream(templatePath);
    InputStream jsonIn = new FileInputStream(jsonPath);
    OutputStream pdfOut = new FileOutputStream(outPdfPath);
    CHtmlConverter.CErrorCode error = CHtmlConverter.RenderAndConvertToPdf(htmlIn, jsonIn, pdfOut);

} catch (Exception e) {

}
```

Use content String:

```
String getFileContent(String path) {
        StringBuilder content = new StringBuilder();
        try (BufferedReader br = new BufferedReader(new InputStreamReader(new FileInputStream(path))
            )) {
            String line;
            while ((line = br.readLine()) != null) {
                content.append(line).append(System.lineSeparator());
            }
```

```
            return content.toString();
        } catch (IOException e) {
            return "";
        }
    }

String templatePath = "/test/template_1.html";
String jsonPath = "/test/data1.json";
String outPdfPath = "/test/outPdf.pdf";
try {
    String templateStr = getFileContent(templatePath);
    String jsonStr = getFileContent(jsonPath);
    OutputStream pdfOut = new FileOutputStream(outPdfPath);
    CHtmlConverter.CErrorCode error = CHtmlConverter.RenderAndConvertToPdf(templateStr, jsonStr, pdf
        Out);
} catch (Exception e) {

}
```

Use File：

```
String templatePath = "/test/template_1.html";
String jsonPath = "/test/data1.json";
String outPdfPath = "/test/outPdf.pdf";
try {
    File templateFile = new File(templatePath);
    File jsonFile = new File(jsonPath);
    File pdfFile = new File(outPdfPath);
    CHtmlConverter.CErrorCode error = CHtmlConverter.RenderAndConvertToPdf(templateFile, jsonFile, p
        dfFile);
} catch (Exception e) {

}
```

3.2.3 Convert HTML to PDF

This method of generating PDF does not require templates or JSON data. It only needs a standard HTML file or a standard HTML string.

```
String htmlPath = "/test/convert.html";
String baseUrl = "/test/";
String outputPdf = "/test/convert.pdf";
int pageWidth = 842;
int pageHeight = 596;
CConverterProperties properties = new CConverterProperties();
CSize size = new CSize(pageWidth, pageHeight);
properties.setPageSize(size);
properties.setBaseUri(baseUrl);
CFontProvider fontProvider = new CFontProvider();
properties.setFontProvider(fontProvider);
CHtmlConverter.CErrorCode id = CHtmlConverter.ConvertToPdf(htmlPath, outputPdf, properties);
```

The HTML can be a content String, InputStream, File object, or file path. The generated PDF can be an OutputStream, File object, or file path. The previous example used file paths, while the following are examples using other types.

Use InputStream and OutputStream:

```
String htmlPath = "/test/convert_1.html";
String outPdfPath = "/test/outPdf.pdf";
try {
    InputStream htmlIn = new FileInputStream(htmlPath);
    OutputStream pdfOut = new FileOutputStream(outPdfPath);
    CHtmlConverter.CErrorCode error = CHtmlConverter.ConvertToPdf(htmlIn, pdfOut);
} catch (Exception e) {
```

```
}
```

Use Content String:

```
String htmlPath = "/test/convert_1.html";
String outPdfPath = "/test/outPdf.pdf";
try {
    String htmlStr = getFileContent(htmlPath);
    OutputStream pdfOut = new FileOutputStream(outPdfPath);
    CHtmlConverter.CErrorCode error = CHtmlConverter.ConvertToPdf(htmlStr, pdfOut);
} catch (Exception e) {

}
```

Use File:

```
String htmlPath = "/test/convert_1.html";
String outPdfPath = "/test/outPdf.pdf";
try {
    File htmlFile = new File(htmlPath);
    File pdfFile = new File(outPdfPath);
    CHtmlConverter.CErrorCode error = CHtmlConverter.ConvertToPdf(htmlFile, pdfFile);
} catch (Exception e) {

}
```

3.2.4 Page Margins

Version 1.3.0 adds `setPageMargins(top, right, bottom, left)` and `getPageMargins()` to `CConverterProperties`. Margins are measured in **CSS pixels** and repeat on every page. Page width and height passed to `setPageSize()` are measured in **points**. Explicit margins in the document's CSS `@page` rules take precedence over these property values.

```
import CPDFGeneration.CConverterProperties;
import CPDFGeneration.CHtmlConverter;

CConverterProperties properties = new CConverterProperties();
try {
    properties.setPageMargins(48, 40, 48, 40);
    int[] margins = properties.getPageMargins();
    System.out.println("Top margin: " + margins[0] + " CSS px");
    CHtmlConverter.CErrorCode status = CHtmlConverter.ConvertToPdf(
            "input.html", "margins.pdf", properties);
    if (status != CHtmlConverter.CErrorCode.HTML_CONVERTER_SUCCESS) {
        throw new IllegalStateException("HTML conversion failed: " + status);
    }
} finally {
    properties.release();
}
```

All four margins must be nonnegative. `getPageMargins()` returns a new array in top/right/bottom/left order; changing that array does not update the properties. A newly created `CConverterProperties` has zero margins. Email conversion without a properties object uses its own defaults, described in Section 3.4.

Keep the properties and any assigned font provider valid throughout conversion. Release the properties after use, as shown above.

3.2.5 Headers and Footers

Use `CHtmlHeaderFooterOptions` with `setHeaderFooterOptions()` to render separate HTML templates in the top and bottom page-margin areas. Set `displayHeaderFooter` to `true`, and reserve enough top/bottom margin for the templates. A template that exceeds its available width or height causes conversion to fail; increase the margin or reduce the template size.

The following example treats the first physical page as a cover page: it hides that page's templates and starts numbering the second physical page at 1.

```
import CPDFGeneration.CConverterProperties;
import CPDFGeneration.CHtmlConverter;
import CPDFGeneration.CHtmlHeaderFooterOptions;

CConverterProperties properties = new CConverterProperties();
try {
    properties.setPageMargins(48, 40, 48, 40);
    CHtmlHeaderFooterOptions options = new CHtmlHeaderFooterOptions();
    options.displayHeaderFooter = true;
    options.headerTemplate = "<div style='font-size:10px'>"
            + "<span class='title'></span></div>";
    options.footerTemplate = "<div style='font-size:10px;text-align:right'>"
            + "<span class='date'></span> | "
            + "<span class='pageNumber'></span>/<span class='totalPages'></span></div>";
    options.title = "Report";
    options.skipLeadingTemplatePages = 1;
    options.skipLeadingNumberedPages = 1;
    properties.setHeaderFooterOptions(options);

    CHtmlHeaderFooterOptions saved = properties.getHeaderFooterOptions();
    System.out.println("Headers/footers enabled: " + saved.displayHeaderFooter);
    CHtmlConverter.CErrorCode status = CHtmlConverter.ConvertToPdf(
            "input.html", "header-footer.pdf", properties);
    if (status != CHtmlConverter.CErrorCode.HTML_CONVERTER_SUCCESS) {
        throw new IllegalStateException("HTML conversion failed: " + status);
    }
} finally {
    properties.release();
}
```

Field	Default	Description
`displayHeaderFooter`	`false`	Enable or disable header/footer rendering.
`headerTemplate`	Empty string	Header HTML; an empty string selects the default header.
`footerTemplate`	Empty string	Footer HTML; an empty string selects the default footer.
`baseUri`	Empty string	Base URI for template resources; empty uses the body resource directory.
`title`	Empty string	Displayed title; empty uses the body's HTML title.
`url`	Empty string	Displayed URL; supply it when converting HTML without a source URL.
`date`	Empty string	Displayed date; empty uses the local date/time of the conversion.

Field	Default	Description
`skipLeadingTemplatePages`	`0`	Hide templates on the first N physical pages without changing numbering.
`skipLeadingNumberedPages`	`0`	Exclude the first N physical pages from numbering and template rendering; subsequent pages start at 1.

Both skip counts must be nonnegative. When both are set, templates are hidden on the first `max(skipLeadingTemplatePages, skipLeadingNumberedPages)` physical pages, and numbering is offset only by `skipLeadingNumberedPages`.

Templates support elements with the CSS classes `date`, `title`, `url`, `pageNumber` and `totalPages`, for example `<span class='pageNumber'></span>`. Page counters also support the text placeholders `{{pageNumber}}` and `{{totalPages}}`. `totalPages` excludes the pages skipped by `skipLeadingNumberedPages`.

An empty template selects the built-in template. To leave a header or footer explicitly blank, use `<div></div>`. Templates use the renderer's supported HTML/CSS and do not execute JavaScript.

`setHeaderFooterOptions()` copies the configuration; changing the supplied object later does not change the properties. `getHeaderFooterOptions()` returns an independent copy that must be applied again to update the configuration. These settings can also be supplied to email PDF conversion through `CConverterProperties`.

3.3 Generate PDF Using Layout API

3.3.1 Introduction

ComPDF Generation SDK provides a powerful Layout API that allows you to directly create and control every element of a PDF document through code. Unlike the HTML to PDF approach, the Layout API provides more fine-grained control, enabling you to build PDF documents entirely from scratch programmatically.

Main Advantages:

- **Complete Programmatic Control**: Precisely control the position, size, and style of each element in the document through code.
- **Flexible Layout System**: Supports block-level elements, inline elements, and absolute positioning.
- **Rich Element Types**: Text, paragraphs, tables, lists, images, separators, etc.
- **Style Management**: Supports reusable styles to achieve consistent document appearance.
- **Box Model Support**: Full support for margin, padding, border, etc.
- **Flowing Layout**: Automatically handles pagination and content reflow.

Applicable Scenarios:

- Scenarios requiring precise control of document layout
- Dynamic generation of structurally complex documents
- Complex conditional logic needed to determine document content
- Deep integration with existing code logic

3.3.2 Core Concepts

Document Structure

The Layout API organizes document content using a tree structure:

```
CDocument (Document)
  ├─ CPage (Page)
  │   ├─ CDiv (Container)
  │   │   ├─ CParagraph (Paragraph)
  │   │   ├─ CTable (Table)
  │   │   ├─ CList (List)
  │   │   ├─ CImage (Image)
  │   │   ├─ CText (Text)
  │   │   └─ CLineSeparator (Separator)
  │   └─ ... (Other elements)
  └─ CStyle (Style)
```

Element Types

1. Container Elements

- **CDocument**: Root container of the PDF document, can contain multiple pages.
- **CPage**: Represents a page of the PDF, can contain various layout elements.
- **CDiv**: Generic container element, similar to div in HTML, used to organize content.
- **CParagraph**: Paragraph container, supports line spacing and first-line indentation.
- **CTable**: Table element, supports cell merging, headers, and footers.
- **CCell**: Table cell.
- **CList**: List container, supports ordered and unordered lists.
- **CListItem**: List item.

2. Content Elements

- **CText**: Text element, can set font, color, size, etc.
- **CImage**: Image element, supports various scaling modes.

3. Other Elements

- **CLineSeparator**: Horizontal separator line.
- **CStyle**: Reusable style definition.

Unit System

The Layout API uses the **CUnitValue** structure to represent dimension values, supporting two units:

```
// Point unit - 1 point = 1/72 inch
CUnitValue pointValue = CUnitValue.createPointValue(100f);

// Percentage unit
CUnitValue percentValue = CUnitValue.createPercentValue(50f);
```

Color System

Use the **CColor** structure to define RGBA colors:

```
// Create color using RGBA values
CColor color = new CColor(255, 0, 0, 255); // Red, fully opaque

// Use predefined colors
CColor black = CColor.Black;
CColor white = CColor.White;
CColor red = CColor.Red;
```

3.3.3 Quick Start

Creating Your First PDF Document

```
import CPDFGeneration.layout.*;

public class Example {
    public static void main(String[] args) {
        // Create a document
        CDocument document = new CDocument();
        // Create a page (A4 size: 595x842 points)
        CPage page = document.createPage(595, 842);
        // Create a div container
        CDiv div = page.createDivElement();
        // Add text
        CText text = div.createTextElement("Hello, PDF Generation!");
        text.setFontSize(24);
        text.setFontColor(CColor.BLACK);
        // Save to file
        document.saveToFile("output.pdf");
    }
}
```

3.3.4 Text and Paragraphs

Creating Text

```
// Create text inside the div
CText text = div.createTextElement("This is a piece of text");

// Set font properties
text.setFontFamily("Arial");
text.setFontSize(14);
text.setFontColor(new CColor(0, 0, 0, 255));
text.setFontWeight(CFontWeight.BOLD);
text.setFontStyle(CFontStyle.ITALIC);

// Set text decoration
text.setUnderline(CColor.BLACK);

// Set character and word spacing
text.setCharacterSpacing(0.5f);
text.setWordSpacing(2.0f);
```

Creating Paragraphs

```
CDiv div = document.createPage(800, 600).createDivElement();
// Create an empty paragraph
CParagraph paragraph0 = div.createParagraphElement();

// Or create a paragraph with initial text
CParagraph paragraph = div.createParagraphElement("Paragraph text content");

// Add text runs to the paragraph
CText text1 = paragraph.createTextElement("First part of the text");
CText text2 = paragraph.createTextElement("Second part of the text");
text2.setFontWeight(CFontWeight.BOLD);

// Set line spacing
paragraph.setFixedLeading(20f); // Fixed line spacing
// Or
paragraph.setMultipliedLeading(1.5f); // Relative line spacing (1.5x)

// Set first-line indent
paragraph.setFirstLineIndent(32f);

// Set text alignment
paragraph.setTextAlignment(CTextAlignment.JUSTIFIED);
```

Text Alignment Options

```
element.setTextAlignment(CTextAlignment.Left);    // Align left
element.setTextAlignment(CTextAlignment.Center);  // Center align
element.setTextAlignment(CTextAlignment.Right);   // Align right
element.setTextAlignment(CTextAlignment.Justify); // Justify
```

3.3.5 Tables

Creating Basic Tables

```
// Method 1: Create a table with a specified number of columns (equal-width columns)
CTable table = div.createTableElement(3);

// Method 2: Create a table with specified column widths (in points)
float[] columnWidths = new float[] { 100, 150, 100 };
CTable table1 = div.createTableElement(columnWidths);

// Method 3: Create a table with specified column widths (supports percentages)
CUnitValue[] columnWidths1 = new CUnitValue[] {
    CUnitValue.CreatePercentValue(30),
    CUnitValue.CreatePercentValue(40),
    CUnitValue.CreatePercentValue(30)
};
CTable table2 = div.createTableElement(columnWidths1);

// Use the full available width
table2.useAllAvailableWidth();
```

Adding Table Content

```
// Start a new row
table.startNewRow();

// Create cells
CCell cell1 = table.createCell(1, 1);
cell1.createTextElement("Cell 1");

CCell cell2 = table.createCell(1, 1);
cell2.createTextElement("Cell 2");

// Create a cell that spans multiple rows and columns
CCell cell3 = table.createCell(2, 2); // rowspan, colspan
cell3.createTextElement("Span 2 rows and 2 columns");
```

Table Headers and Footers

```
// Create a table header
table.startNewHeaderRow();
CCell headerCell1 = table.createHeaderCell(1, 1);
headerCell1.createTextElement("Column 1");
CCell headerCell2 = table.createHeaderCell(1, 1);
headerCell2.createTextElement("Column 2");

// Create a table footer
table.startNewFooterRow();
CCell footerCell1 = table.createFooterCell(1, 1);
footerCell1.createTextElement("Total");
CCell footerCell2 = table.createFooterCell(1, 1);
footerCell2.createTextElement("1000");

// Create a table caption (title)
CDiv caption = table.createCaption();
caption.createTextElement("Sales Data Table");
```

Set Table Styling

```
// Set the border collapse mode
table.setBorderCollapse(CBorderCollapseType.COLLAPSE);

// Set border spacing (only takes effect in Separate mode)
table.setBorderCollapse(CBorderCollapseType.SEPARATE);
table.setHorizontalBorderSpacing(5f);
table.setVerticalBorderSpacing(5f);

// Set the cell border
CBorder border = new CBorder(
    CUnitValue.CreatePointValue(1),
    CBorder.CBorderType.SOLID,
    CColor.BLACK
);
cell.setBorder(border);

// Set the cell background color
cell.setBackgroundColor(new CColor(240, 240, 240, 255));

// Set cell alignment
cell.setTextAlignment(CTextAlignment.CENTER);
cell.setVerticalAlignment(CVerticalAlignment.MIDDLE);
```

3.3.6 Lists

Unordered Lists

```
// Create an unordered list
CList list = div.createListElement();

// Set the list bullet style
list.setListSymbol(CListSymbolType.DISC);     // Filled circle
// list.setListSymbol(CListSymbolType.Circle); // Hollow circle
// list.setListSymbol(CListSymbolType.Square); // Square

// Or use a custom text symbol
list.setListSymbol("→");

// Add list items
CListItem item1 = list.createListItemElement("Item 1");
CListItem item2 = list.createListItemElement("Item 2");
CListItem item3 = list.createListItemElement("Item 3");
```

Ordered Lists

```
// Create an ordered list
CList list = div.createList();

// Set the numbering style
list.setListSymbol(CListNumberType.DECIMAL);           // 1, 2, 3, ...
// list.setListSymbol(CListNumberType.ENGLISH_LOWER);  // a, b, c, ...
// list.setListSymbol(CListNumberType.ENGLISH_UPPER);  // A, B, C, ...
// list.setListSymbol(CListNumberType.ROMAN_LOWER);    // i, ii, iii, ...
// list.setListSymbol(CListNumberType.ROMAN_UPPER);    // I, II, III, ...

// Set the starting index
list.setItemStartIndex(5); // Start numbering from 5

// Add list items
list.createListItemElement("Item 1");
list.createListItemElement("Item 2");
```

Nested Lists

```
CList outerList = div.createListElement();
outerList.setListSymbol(CListNumberType.DECIMAL);

CListItem item1 = outerList.createListItemElement("Main item 1");

// Create a nested list inside the list item
CList innerList = item1.createListElement();
innerList.setListSymbol(CListSymbolType.CIRCLE);
innerList.createListItemElement("Sub-item 1.1");
innerList.createListItemElement("Sub-item 1.2");

CListItem item2 = outerList.createListItemElement("Main item 2");
```

3.3.7 Images

Adding Images

```
// Create an image from a file path
CImage image = div.createImageElement("path/to/image.png");

// Set the image size (in points)
image.setWidth(300);
image.setHeight(200);

// Or use a percentage value
image.setWidth(CUnitValue.CreatePercentValue(50));
```

Image Scaling

```
// Scale proportionally
image.scale(0.5f, 0.5f); // Scale to 50%

// Fit to a specified size (keep aspect ratio)
image.scaleToFit(400, 300);

// Scale to an exact size (may distort the image)
image.scaleAbsolute(400, 300);

// Auto scaling
image.setAutoScale(true);
image.setAutoScaleWidth(true);
image.setAutoScaleHeight(true);
```

Image Positioning

```
// Absolute positioning
image.setAbsolutePosition(100, 100);

// Set margin
image.setMargin(10);
image.setMargins(10, 20, 10, 20); // Top, Right, Bottom, Left

// Set padding
image.setPadding(5);
```

3.3.8 Style Management

Creating and Using Styles

```
// Create a style
CStyle headingStyle = document.createStyle();

// Set style properties (supports method chaining)
headingStyle
    .setFontSize(18)
    .setFontWeight(CFontWeight.BOLD);

headingStyle.setMarginBottom(10)
    .setTextAlignment(CTextAlignment.CENTER);

// Apply the style to an element
CText heading = div.createTextElement("Heading text");
heading.addStyle(headingStyle);

// An element can also have multiple styles
CStyle colorStyle = document.createStyle();
colorStyle.setFontColor(CColor.RED);
heading.addStyle(colorStyle);
```

3.3.9 Box Model

Margin

```
// Set uniform margin
div.setMargin(20);

// Set individual margins
div.setMargins(10, 20, 10, 20); // Top, Right, Bottom, Left

// Set individual sides
div.setMarginTop(10);
div.setMarginRight(20);
div.setMarginBottom(10);
div.setMarginLeft(20);
```

Padding

```
// Set uniform padding
div.setPadding(15);

// Set individual paddings
div.setPaddings(10, 15, 10, 15);

// Set individual sides
div.setPaddingTop(10);
div.setPaddingRight(15);
div.setPaddingBottom(10);
div.setPaddingLeft(15);
```

Border

```
// Create a border
border.setType(CBorder.CBorderType.SOLID);  // Solid, Dashed, Dotted, Double
border.setColor(new CColor(0, 0, 0, 255));

// Apply to all sides
div.setBorder(border);

// Or set each side separately
div.setBorderTop(border);
div.setBorderRight(border);
div.setBorderBottom(border);
div.setBorderLeft(border);
```

Border Radius

```
// Create a uniform border radius
CBorderRadius radius = new CBorderRadius(10f);
div.setBorderRadius(radius);

// Create an elliptical border radius
CBorderRadius radius2 = new CBorderRadius(10f, 5f);
div.setBorderRadius(radius2);

// Set individual corners
CBorderRadius topLeft = new CBorderRadius(10f);
div.setBorderTopLeftRadius(topLeft);
```

Width and Height

```
// Set fixed dimensions
div.setWidth(400);
div.setHeight(300);

// Use percentage
div.setWidth(CUnitValue.CreatePercentValue(50));

// Set max/min dimensions
div.setMaxWidth(600);
div.setMinWidth(200);
div.setMaxHeight(400);
div.setMinHeight(100);
```

3.3.10 Positioning

Relative Positioning (Default)

Elements are arranged according to normal document flow, which is the default behavior.

```
CDiv div1 = page.createDivElement();
div1.createTextElement("First div");

CDiv div2 = page.createDivElement();
div2.createTextElement("Second div"); // Will appear below div1
```

Absolute Positioning

```
// Create absolutely positioned element
CDiv div = page.createDivElement();
div.setAbsolutePosition(
    100,    // left: 100 points from page left edge
    200,    // bottom: 200 points from page bottom edge
    300     // width: Width 300 points
);

// Or use percentage width
div.setAbsolutePosition(
    100,
    200,
    CUnitValue.CreatePercentValue(50)
);
```

3.3.11 Background

Set Background Color

```
div.setBackgroundColor(new CColor(240, 240, 240, 255));
```

Set Background Image

```
// Create a background image
CBackgroundImage bgImage = new CBackgroundImage();

// Set a single image source
bgImage.setSource("path/to/background.png");

ArrayList<String> sources = new ArrayList<>();
sources.add("path/to/image1.png");
sources.add("path/to/image2.png");
// Or set multiple image sources
bgImage.setSources(sources);

// Apply the background image
div.setBackgroundImage(bgImage);
```

3.3.12 Separators

```
// Create separator
CLineSeparator separator = div.createLineSeparatorElement();

// Set line style
separator.setLineWidth(2f);
separator.setLineType(CLineSeparator.CPDFLineType.SOLID); // Solid, Dashed, Dotted
separator.setLineColor(new CColor(200, 200, 200, 255));

// Set separator width (uses all available width by default)
separator.setWidth(CUnitValue.CreatePercentValue(80));
```

3.3.13 Font Management

```
// Create font provider
CFontProvider fontProvider = new CFontProvider();

// Add font directory
fontProvider.addDirectory("path/to/fonts");

// Add system fonts
fontProvider.addSystemFonts();

// Apply font provider to document
document.setFontProvider(fontProvider);

// Use fonts in elements
div.setFontFamily("Arial");
```

3.3.14 Complete Examples

Creating an Invoice Document

```
import CPDFGeneration.layout.*;

public class InvoiceExample {
    public static void generateInvoice() {
        // Create a document
        CDocument document = new CDocument();

        // Create an A4 page
        CPage page = document.createPage(595, 842);

        // Create a page container
        CDiv container = page.createDivElement();
        container.setPadding(40);

        // Create a title style
        CStyle titleStyle = document.createStyle();
        titleStyle
            .setFontSize(24)
            .setFontWeight(CFontWeight.BOLD);

        titleStyle.setMarginBottom(20)
            .setTextAlignment(CTextAlignment.CENTER);

        // Add the title
        CText title = container.createTextElement("Sales Invoice");
        title.addStyle(titleStyle);

        // Add invoice information
        CDiv infoSection = container.createDivElement();
        infoSection.setMarginBottom(20);

        CParagraph invoiceNo = infoSection.createParagraphElement("Invoice No.: INV-2024-001");
        invoiceNo.setFontSize(12);

        CParagraph date = infoSection.createParagraphElement("Date: 2024-01-15");
        date.setFontSize(12);

        // Create the product table
        CTable table = container.createTableElement(4);
        table.useAllAvailableWidth();
        table.setMarginBottom(20);

        // Table header
        table.startNewHeaderRow();
        table.createHeaderCell().createTextElement("Product Name");
        table.createHeaderCell().createTextElement("Quantity");
        table.createHeaderCell().createTextElement("Unit Price");
        table.createHeaderCell().createTextElement("Amount");

        // Table data
        String[] products = { "Product A", "Product B", "Product C" };
        int[] quantities = { 2, 5, 3 };
        double[] prices = { 100.0, 50.0, 75.0 };

        for (int i = 0; i < products.length; i++) {
            table.startNewRow();
            table.createCell().createTextElement(products[i]);
            table.createCell().createTextElement(String.valueOf(quantities[i]));
            table.createCell().createTextElement("¥" + prices[i]);
            table.createCell().createTextElement("¥" + (quantities[i] * prices[i]));
        }
```

```
        // Calculate the total
        double total = 0;
        for (int i = 0; i < products.length; i++) {
            total += quantities[i] * prices[i];
        }

        // Table footer - Total
        table.startNewFooterRow();
        CCell totalLabelCell = table.createFooterCell(1, 3); // rowspan, colspan
        totalLabelCell.createTextElement("Total");
        totalLabelCell.setTextAlignment(CTextAlignment.RIGHT);
        totalLabelCell.setFontWeight(CFontWeight.BOLD);

        CCell totalAmountCell = table.createFooterCell();
        totalAmountCell.createTextElement("¥" + total);
        totalAmountCell.setFontWeight(CFontWeight.BOLD);

        // Add notes
        CParagraph notes = container.createParagraphElement("Notes: Please make payment within 30 da
            ys.");
        notes.setFontSize(10);
        notes.setFontColor(new CColor(100, 100, 100, 255));

        // Save the document
        document.saveToFile("invoice.pdf");
    }
}
```

Creating a Report Document

```
public class ReportExample {
    public void generateReport() {
        CDocument document = new CDocument();

        CPage page = document.createPage(595, 842);
        CDiv container = page.createDivElement();
        container.setPadding(50);

        // Title
        CText title = container.createTextElement("Annual Report");
        title.setFontSize(28);
        title.setFontWeight(CFontWeight.BOLD);
        title.setTextAlignment(CTextAlignment.CENTER);

        // Add sections
        addSection(
            container,
            "1. Overview",
            "This report summarizes the company's major achievements and development direction over
                the past year..."
        );

        // Add a chart
        CDiv chartSection = container.createDivElement();
        chartSection.setMarginTop(20);
        chartSection.setMarginBottom(20);

        CText chartTitle = chartSection.createTextElement("2. Sales Data");
        chartTitle.setFontSize(18);
        chartTitle.setFontWeight(CFontWeight.BOLD);

        CImage chart = chartSection.createImageElement("path/to/chart.png");
        chart.setWidth(400);
        chart.setHeight(300);
        chart.setHorizontalAlignment(CHorizontalAlignment.CENTER);

        // Add a key metrics list
        CText metricsTitle = container.createTextElement("3. Key Metrics");
        metricsTitle.setFontSize(18);
        metricsTitle.setFontWeight(CFontWeight.BOLD);

        CList metricsList = container.createListElement();
        metricsList.setListSymbol(CListSymbolType.DISC);
        metricsList.createListItemElement("Annual revenue growth: 25%");
        metricsList.createListItemElement("Customer satisfaction: 92%");
        metricsList.createListItemElement("Market share increase: 3%");

        document.saveToFile("report.pdf");
    }

    private void addSection(CDiv container, String heading, String content) {
        CParagraph headingPara = container.createParagraphElement(heading);
        headingPara.setFontSize(18);
        headingPara.setFontWeight(CFontWeight.BOLD);
        headingPara.setMarginTop(20);
        headingPara.setMarginBottom(10);

        CParagraph contentPara = container.createParagraphElement(content);
        contentPara.setFontSize(12);
        contentPara.setTextAlignment(CTextAlignment.JUSTIFIED);
        contentPara.setMultipliedLeading(1.5f);
    }
}
```

3.3.15 Best Practices

Resource Management

```
CDocument document = new CDocument();
  CBorder border = new CBorder(1f);
  CBorderRadius radius = new CBorderRadius(5f)
  // Use these resources
  document.saveToFile("output.pdf");
```

Style Reuse

```
// Create reusable styles
CStyle bodyTextStyle = document.createStyle();
bodyTextStyle
    .setFontSize(12)
    .setFontFamily("Arial");
bodyTextStyle.setMarginBottom(10);

CStyle headingStyle = document.createStyle();
headingStyle
    .setFontSize(18)
    .setFontWeight(CFontWeight.Bold);
headingStyle.setMarginTop(15)
    .setMarginBottom(10);

// Apply to multiple elements
paragraph1.addStyle(bodyTextStyle);
paragraph2.addStyle(bodyTextStyle);
heading1.addStyle(headingStyle);
heading2.addStyle(headingStyle);
```

Performance Optimization

```
// 1. When creating elements in bulk, consider pre-creating styles
CStyle itemStyle = document.createStyle();
itemStyle.setFontSize(12);
itemStyle.setMarginBottom(5);

for (int i = 0; i < 1000; i++) {
    CText text = div.createTextElement("Item " + i);
    text.addStyle(itemStyle); // Reuse style instead of setting each time
}

// 2. Use stream output instead of writing to memory first
try (FileOutputStream fos = new FileOutputStream("large.pdf")) {
    document.saveToFile(fos);
} catch (IOException e) {
    e.printStackTrace();
}
```

Error Handling

```
try {
    CDocument document = new CDocument();
    CPage page = document.createPage(595, 842);

    // Validate the image path
    String imagePath = "path/to/image.png";
    File imageFile = new File(imagePath);
    if (imageFile.exists()) {
        CImage image = page.createDivElement().createImageElement(imagePath);
    } else {
        System.out.println("Image file does not exist: " + imagePath);
    }

    document.saveToFile("output.pdf");

} catch (Exception ex) {
    System.out.println("Error generating PDF: " + ex.getMessage());
    ex.printStackTrace();
}
```

3.3.16 Developer FAQs

Q: How to set page orientation to landscape?

A: Swap width and height values when creating the page:

```
// Portrait A4: 595 x 842
CPage portraitPage = document.createPage(595, 842);

// Landscape A4: 842 x 595
CPage landscapePage = document.createPage(842, 595);
```

Q: How to implement multi-column layout?

A: Use tables or multiple divs with width settings:

```
CTable layout = page.createTableElement(2);
layout.useAllAvailableWidth();
layout.setBorderCollapse(CBorderCollapseType.COLLAPSE);

// Hide borders
CBorder noBorder = new CBorder(0);
layout.setBorder(noBorder);

layout.startNewRow();
CCell leftColumn = layout.createCell();
CCell rightColumn = layout.createCell();

// Add content to each column
leftColumn.createTextElement("Left column content");
rightColumn.createTextElement("Right column content");
```

3.4 Convert Email to HTML and PDF

3.4.1 Formats and Initialization

Version 1.3.0 introduces the `CEmailConverter` API for EML/MIME messages and Outlook MSG files. It supports file paths, `File` objects, byte arrays and input streams, and can generate an HTML package or a PDF containing the email body and an optional attachment list.

Use the matching version of the JAR and native library. Before running the PDF examples below, initialize the SDK with a valid license as described in Section 2.3. HTML parsing and HTML-package export do not require a license, but the native library must still be loaded; for an HTML-only application, `System.load()` can load the matching native library by its absolute path.

`CEmailFormat` provides `EMAIL_FORMAT_AUTO`, `EMAIL_FORMAT_EML` and `EMAIL_FORMAT_MSG`. AUTO identifies MSG from its Compound File Binary signature and otherwise parses the input as EML; it does not rely on the filename extension. Pass binary MSG data directly rather than converting it to a Java string.

3.4.2 Convert an Email File to PDF

```
import CPDFGeneration.CEmailConverter;
import CPDFGeneration.CEmailDiagnostic;
import CPDFGeneration.CEmailResult;

try (CEmailResult result = CEmailConverter.ConvertToPdf("message.eml", "message.pdf")) {
    if (!result.isSuccess()) {
        for (int i = 0; i < result.GetDiagnosticCount(); i++) {
            CEmailDiagnostic diagnostic = result.GetDiagnostic(i);
            System.err.println(diagnostic.code + ": " + diagnostic.detail);
        }
        throw new IllegalStateException("Email conversion failed: " + result.GetStatus(),
                result.GetLastCallbackException());
    }
    System.out.println("Converted format: " + result.GetFormat());
}
```

The same example accepts an MSG input by replacing `message.eml` with a MSG path. The output PDF must not already exist: path output preserves existing files and removes a partial file created by the operation if it fails. Always close the returned `CEmailResult`, including on failure. Both `EMAIL_SUCCESS` and `EMAIL_SUCCESS_WITH_WARNINGS` are successful; `isSuccess()` accepts either.

The PDF contains the selected email body and, when enabled, an attachment list. It does not embed attachment files or render attachment contents as additional pages. Attachment download links are omitted from the PDF; use HTML-package export or attachment data access when you need the files.

3.4.3 Export HTML and Write PDF in Separate Steps

Parse once with `ConvertToHtml()`, then export an offline HTML package or write a PDF from the retained content. This example also queries the selected primary body's provenance and applies custom page margins.

```
import CPDFGeneration.CConverterProperties;
import CPDFGeneration.CEmailBodySource;
import CPDFGeneration.CEmailConverter;
import CPDFGeneration.CEmailResult;
import CPDFGeneration.CEmailStatus;

CConverterProperties properties = new CConverterProperties();
try {
    properties.setPageMargins(48, 40, 48, 40);
    try (CEmailResult result = CEmailConverter.ConvertToHtml("message.msg")) {
        if (!result.isSuccess()) {
            throw new IllegalStateException("Email parsing failed: " + result.GetStatus());
        }
        CEmailBodySource body = result.GetSelectedBodySource();
        if (body.status.isSuccess()) {
            System.out.println(body.mediaType + " / " + body.origin);
        }
        CEmailStatus exported = result.ExportHtml("mail-package");
        if (!exported.isSuccess()) {
            throw new IllegalStateException("HTML export failed: " + exported);
        }
```

```
        CEmailStatus written = result.WritePdf("message.pdf", properties);
        if (!written.isSuccess()) {
            throw new IllegalStateException("PDF write failed: " + written,
                    result.GetLastCallbackException());
        }
    }
} finally {
    properties.release();
}
```

Use a new output directory for the HTML package. `ExportHtml()` writes `index.html`, body resources and enabled attachments without overwriting existing output files. Failure removes files created by that export, while preserving caller data. `GetHtml()` returns only the UTF-8 HTML string, which may contain relative resource paths; use `ExportHtml()` to obtain a complete offline package.

Check the status returned by each export/write operation. `GetStatus()` describes the latest operation and can replace the original conversion status. Each new native output operation recalculates status and diagnostics from the initial HTML conversion state. Previously obtained Java snapshots remain valid.

PDF, license or output failures can retain successfully generated HTML, resources and attachments, allowing a later export or PDF retry. An input parsing failure does not guarantee that HTML exists. `GetSelectedBodySource().status` describes the provenance query only and does not change the result's operation status. It identifies the selected primary body, not every displayed multipart segment; its source IDs are opaque within that result. The returned `CEmailBodySource` contains `bodyId`, `sourceId`, `mediaType` and `origin`; the media type is `text/plain` or `text/html`, and the origin is `original`, `recovered`, `fallback` or `generated`.

When PDF properties are `null`, email conversion uses A4 with top/bottom margins of 32 CSS px and left/right margins of 40 CSS px. A supplied properties object preserves its configured page size, margins (including zero), font provider and header/footer settings. The operation does not modify the caller's properties or base URI. HTML-preview padding is not added to PDF page margins. See Sections 3.2.4 and 3.2.5 for print configuration.

3.4.4 Options and Cancellation

Pass `null` options for defaults, or create a `CEmailOptions` object. Options are copied when conversion begins; do not modify them concurrently while the copy is being made.

Field	Default	Description
`flags`	`7`	Include headers, show the attachment list and enable attachment export.
`maxInputBytes`	100 MiB	Maximum input size in bytes.
`maxDecodedBytes`	256 MiB	Decoded-content byte budget.
`maxHtmlBytes`	64 MiB	Generated HTML byte budget.
`cancellationCallback`	`null`	Optional `CEmailCancelCallback` returning `true` to request cancellation.

Combine `CEmailFlags` values with bitwise OR:

Flag	Effect
`EMAIL_INCLUDE_HEADERS`	Display the email header summary. Enabled by default.

Flag	Effect
`EMAIL_INCLUDE_ATTACHMENT_LIST`	Display an attachment list. Enabled by default.
`EMAIL_EXPORT_ATTACHMENTS`	Export available attachment files with the HTML package. Enabled by default.
`EMAIL_INLINE_RESOURCES`	Inline body resources in the generated HTML.
`EMAIL_RECOVER_EML`	Enable bounded recovery of malformed EML; inspect diagnostics for fallback or skipped content.
`EMAIL_INCLUDE_BCC`	Display Bcc, which is hidden by default.

```
import CPDFGeneration.CEmailConverter;
import CPDFGeneration.CEmailFlags;
import CPDFGeneration.CEmailFormat;
import CPDFGeneration.CEmailOptions;
import CPDFGeneration.CEmailResult;
import java.util.concurrent.atomic.AtomicBoolean;

CEmailOptions options = new CEmailOptions();
options.flags |= CEmailFlags.EMAIL_INLINE_RESOURCES;
options.maxInputBytes = 20L * 1024 * 1024;
AtomicBoolean cancelled = new AtomicBoolean(false);
options.cancellationCallback = cancelled::get;
try (CEmailResult result = CEmailConverter.ConvertToHtml(
        "message.eml", CEmailFormat.EMAIL_FORMAT_AUTO, options)) {
    if (!result.isSuccess()) {
        throw new IllegalStateException("Email parsing failed: " + result.GetStatus(),
                result.GetLastCallbackException());
    }
    System.out.println("HTML characters: " + result.GetHtml().length());
}
```

Byte budgets are nonnegative; zero is a real zero limit, not unlimited. Negative budgets throw `IllegalArgumentException` when passed to a converter. Unknown flag bits produce `EMAIL_ERROR_INVALID_PARAMETER`. Internal parser/structural limits also apply, even when the configured byte budgets have not been exceeded.

Cancellation is cooperative, checked between phases and during callback I/O. It does not interrupt a parser or layout operation already in progress and does not provide a hard timeout. The callback runs synchronously on the calling Java thread and must return promptly. Another thread can request cancellation by setting the example's `cancelled` flag to `true`.

The result retains the callback for subsequent `ExportHtml()` and `WritePdf()` calls until the result is closed. Changing the original options object does not replace that retained callback. A callback exception is treated as cancellation and is available through `GetLastCallbackException()`. Do not reenter SDK operations, close the active result, release its properties or synchronously wait for another PDF task from the callback.

3.4.5 Stream and Memory Input

For an input stream, supply the number of email bytes to read from its current position. Do not use `InputStream.available()` as the message length. For a file, use `Files.size()` and ensure that the input does not change while it is read.

```
import CPDFGeneration.CEmailConverter;
import CPDFGeneration.CEmailResult;
import java.io.InputStream;
import java.io.OutputStream;
```

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;

Path input = Paths.get("message.eml");
Path output = Paths.get("message-stream.pdf");
long length = Files.size(input);
try (InputStream source = Files.newInputStream(input);
     OutputStream destination = Files.newOutputStream(output,
             StandardOpenOption.CREATE_NEW, StandardOpenOption.WRITE);
     CEmailResult result = CEmailConverter.ConvertToPdf(source, length, destination)) {
    if (!result.isSuccess()) {
        throw new IllegalStateException("Stream conversion failed: " + result.GetStatus(),
                result.GetLastCallbackException());
    }
}
```

Short reads are retried. Premature EOF produces `EMAIL_ERROR_READ` with the original `EOFException` retained by the result. SDK operations do not close or flush caller streams; the try-with-resources block above closes them on the caller's behalf. On failure or cancellation, discard partial output, including a file created by the caller for the output stream.

To work entirely in memory:

```
import CPDFGeneration.CEmailConverter;
import CPDFGeneration.CEmailResult;
import java.io.ByteArrayOutputStream;
import java.nio.file.Files;
import java.nio.file.Paths;

byte[] email = Files.readAllBytes(Paths.get("message.msg"));
ByteArrayOutputStream output = new ByteArrayOutputStream();
try (CEmailResult result = CEmailConverter.ConvertToPdf(email, output)) {
    if (!result.isSuccess()) {
        throw new IllegalStateException("Memory conversion failed: " + result.GetStatus(),
                result.GetLastCallbackException());
    }
    byte[] pdf = output.toByteArray();
    System.out.println("PDF bytes: " + pdf.length);
}
```

Memory input and output consume Java heap space. For large messages, prefer file or stream input and avoid copying attachment payloads unless they are needed.

The following overload groups provide both convenience methods and full methods with explicit format/options. PDF methods also accept `CConverterProperties`:

Operation	Input	Output	Additional full-method parameters
`ConvertToHtml`	`String` path or `File`	`CEmailResult`	`CEmailFormat`, `CEmailOptions`
`ConvertToHtml`	`byte[]`	`CEmailResult`	`CEmailFormat`, `CEmailOptions`
`ConvertToHtml`	`InputStream`, `long length`	`CEmailResult`	`CEmailFormat`, `CEmailOptions`
`ConvertToPdf`	`String` path	`String` PDF path	`CEmailFormat`, `CEmailOptions`, `CConverterProperties`
`ConvertToPdf`	`File`	`File` PDF output	`CEmailFormat`, `CEmailOptions`, `CConverterProperties`

Operation	Input	Output	Additional full-method parameters
`ConvertToPdf`	`byte[]`	`OutputStream`	`CEmailFormat`, `CEmailOptions`, `CConverterProperties`
`ConvertToPdf`	`InputStream`, `long length`	`OutputStream`	`CEmailFormat`, `CEmailOptions`, `CConverterProperties`

Convenience methods use AUTO format and default options. Convenience PDF methods also use default email page properties. All converter methods return an owned `CEmailResult` that must be closed.

3.4.6 Result Data and Status Codes

Result method	Purpose
`GetStatus()` / `isSuccess()`	Status of the latest conversion or native output operation.
`GetFormat()`	Input format reported by the parser.
`GetPdfError()`	Underlying `CHtmlConverter.CErrorCode` for PDF/license failures.
`GetHtml()`	HTML string, or an empty string if HTML was not generated.
`GetResourceCount()` / `GetResource(index)`	Body-resource count and metadata/payload snapshots.
`GetAttachmentCount()` / `GetAttachment(index)`	Attachment count and metadata/payload snapshots.
`GetDiagnosticCount()` / `GetDiagnostic(index)`	Error/warning diagnostics, including code, source ID, detail and fallback action.
`GetSelectedBodySource()`	Selected primary body's query status, IDs, media type and origin.
`GetLastCallbackException()`	Captured Java stream/cancellation exception, or `null`.
`ExportHtml(String / File)`	Export an offline HTML package.
`WritePdf(String / File / OutputStream [, properties])`	Generate or retry PDF from retained HTML.

`GetResource()` returns `CEmailResource`; `GetAttachment()` returns `CEmailAttachment`; `GetDiagnostic()` returns `CEmailDiagnostic`. Resources and attachments provide metadata such as media type, relative path and size. Attachments also expose `id`, `originalName`, `displayName`, `hasData`, `hidden`, `referencedByBody` and `nestedMessage`. A metadata-only attachment has `hasData == false` and `getData() == null`, unlike a valid zero-byte attachment.

Collection indices are zero-based; an invalid index throws `IndexOutOfBoundsException`. Metadata and payload snapshots are Java-owned copies that remain valid after the result is closed. Each `getData()` call returns another independent byte-array copy, so large payloads require additional heap space.

`CEmailStatus`	Value	Meaning
`EMAIL_SUCCESS`	`0`	Success without warnings.

CEmailStatus	Value	Meaning
EMAIL_SUCCESS_WITH_WARNINGS	1	Success with warnings or fallback content.
EMAIL_ERROR_INVALID_PARAMETER	-1	Invalid argument, format or option.
EMAIL_ERROR_READ	-2	Input read failure.
EMAIL_ERROR_PARSE	-3	Email parsing failure.
EMAIL_ERROR_HTML	-4	HTML generation failure.
EMAIL_ERROR_WRITE	-5	Output write failure.
EMAIL_ERROR_PDF	-6	PDF rendering failure.
EMAIL_ERROR_LICENSE	-7	PDF license check failed.
EMAIL_ERROR_LIMIT	-8	A byte budget or internal structural limit was exceeded.
EMAIL_ERROR_CANCELLED	-9	Cancellation requested.
EMAIL_ERROR_OUT_OF_MEMORY	-10	Native allocation failure.
EMAIL_ERROR_INTERNAL	-11	Internal error or unknown native status.

Typical conversion failures are reported as statuses. Java-side argument validation, such as negative budgets or a path containing NUL, can throw `IllegalArgumentException`; Java/JNI allocation failures can throw `OutOfMemoryError`. Stream and cancellation exceptions are retained by `GetLastCallbackException()`, and that value is replaced by the next native output operation. A Java validation failure leaves the previous operation state intact.

3.4.7 Resource Lifetime and Concurrency

- Use try-with-resources for every `CEmailResult`. `close()` and `release()` are idempotent; native queries/output after closing throw `IllegalStateException`. `isClosed()` remains available.
- Keep supplied converter properties and their font provider valid and unchanged during conversion. Email operations borrow properties under the same lock used by `release()`; released properties cannot be reused with the email API.
- Independent results can be parsed or exported concurrently. Operations and close on the same Java result are serialized.
- Email PDF rendering is serialized internally. Do not render concurrently through the older HTML API or change global license/font state while PDF rendering is active.
- Respect the callback restrictions in Section 3.4.4 to avoid reentry or waiting on an active conversion.
- Java paths and metadata use UTF-8. Linux JDK 8 also requires a UTF-8 filesystem locale for paths containing non-ASCII characters.

3.4.8 Email Sample

Run the packaged example from the `samples` directory:

```
cd EmailConvertTest
bash RunTest.sh
```

The example reads the fixed EML/MSG inputs directly from `samples/TestFiles/email`, and writes an HTML package, a PDF with headers/footers, a nested-message PDF and a stream-generated PDF to a new `samples/out/email-example-*` directory. On Linux, its script locates and preloads `libjsig.so` from the Java installation on PATH to preserve JVM signal-handler behavior. Cached online-license background refresh remains enabled by default.

`PrintOptionsTest` demonstrates page margins and header/footer configuration. `FamilyHeaderFooterTest` demonstrates cover-page handling and page numbering with `TestFiles/family_html/family_all_pages.html`.

4 Technical support

4.1 Report issues

Thank you for your interest in the ComPDF Generation SDK, an easy-to-use and powerful development solution that integrates high-quality PDF generation capabilities into your applications. If you encounter any technical issues while using the ComPDF Generation SDK for Java, please submit your issue report to the ComPDF Team. The following information will help us resolve your issue:

- ComPDF Generation SDK Product and version.
- Your operating system and IDE version.
- A detailed description of the issue.
- Any other relevant information, such as error screenshots.

4.2 Contact information

Website link：

- Home Page: https://www.compdf.com
- Chinese Home Page: https://www.compdf.com/zh-cn
- Developer Guides: https://www.compdf.com/guides/generation-sdk/java/overview
- API Reference: https://developers.compdf.com/guides/generation-sdk/java/api-reference/

Contact ComPDF：

- Contact sales：https://www.compdf.com/contact-sales
- Technical issues should be reported to：https://www.compdf.com/support
- Email（direct contact）：support@compdf.com

Thanks，

ComPDF team